

MINI UMWAMBI

Vision-guided dive correction: how an onboard gimbal camera measures the aircraft's aiming error against a visual target and reports it to the flight controller — independent of GPS.

July 2026 · Platform: Jetson companion computer + SIYI ZR10 + ArduPilot FCC

GPS tells a UAV where it is; it cannot tell the UAV where the *target* is once the target is a visual object below. During a diving approach — at any dive angle from shallow to near-vertical — this system uses the camera to answer one question, ten times per second: **“how far is the nose off the target, in degrees of yaw and pitch?”** That pair of numbers is the correction the flight controller needs to steer the dive onto the target.

1 System setup and communications

Three devices cooperate. The **ZR10 gimbal camera** sees the target and keeps itself pointed at it. The **companion computer** runs the dashboard: it detects the target, steers the gimbal, and does the angle mathematics. The **flight controller (FCC)** supplies the aircraft's attitude and receives the resulting correction over MAVLink.

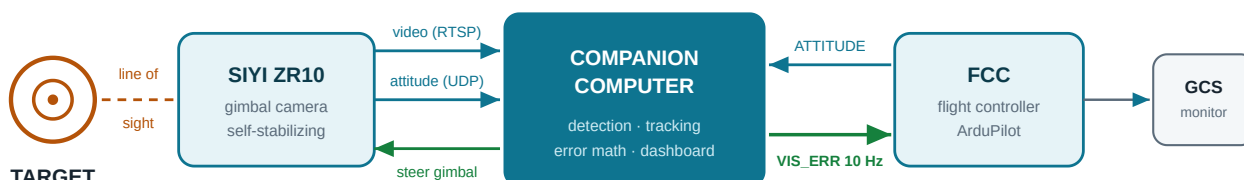
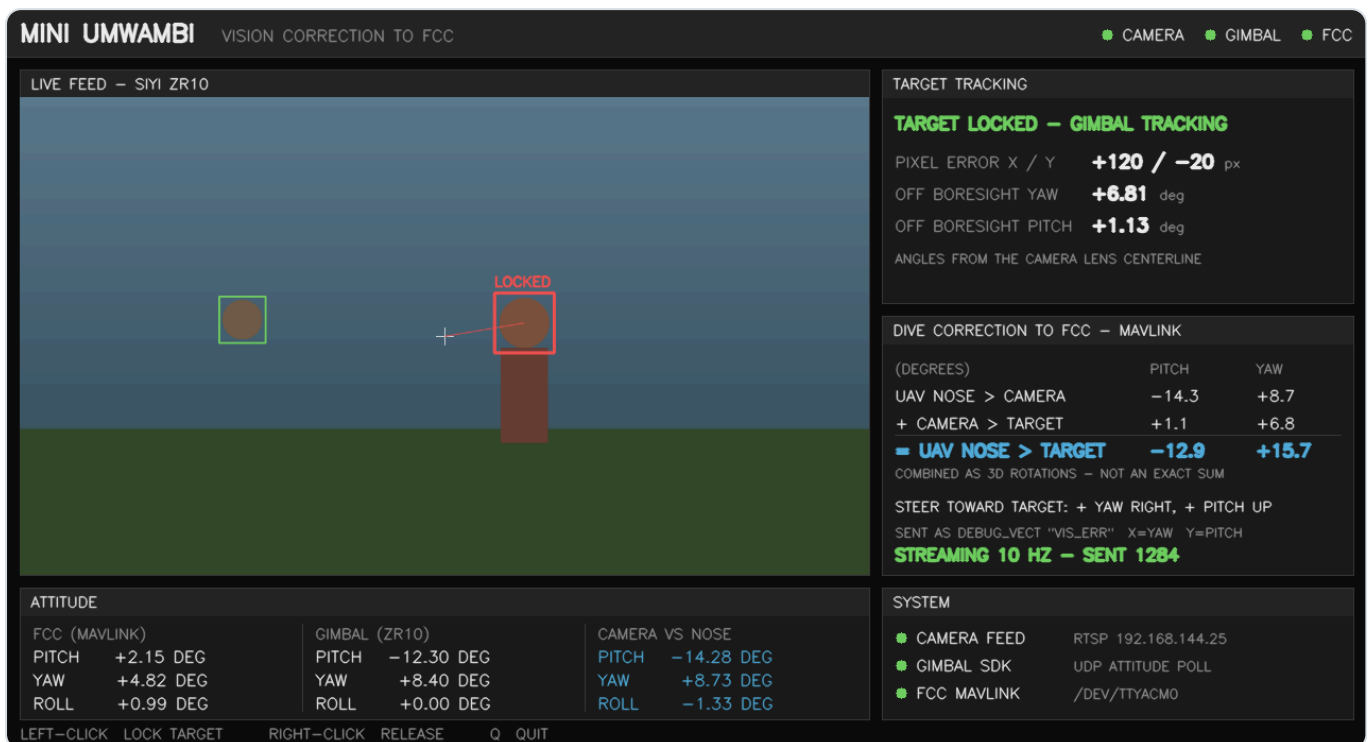


Fig. 1 — Setup and data links. Serial MAVLink carries the aircraft attitude in and the vision correction (`DEBUG_VECT` "VIS_ERR") out; the FCC forwards telemetry to any ground station.

Link	Transport	Content	Rate
Camera → Companion	RTSP over Ethernet	Live video, 1280×720	stream
Gimbal ↔ Companion	SIYI SDK over UDP	Gimbal attitude in; steering rates out	10 Hz
FCC → Companion	MAVLink, serial	<code>ATTITUDE</code> , <code>HEARTBEAT</code>	stream
Companion → FCC	MAVLink, serial	<code>DEBUG_VECT</code> "VIS_ERR": x = yaw error, y = pitch error (deg)	10 Hz

2 What the dashboard displays



- 1 Live Feed (SIYI ZR10).** The camera stream with every detected head boxed in green, the operator-locked target boxed in red, the image center cross (the camera's boresight), and a line from boresight to target — the error you are about to see as numbers.
- 2 Target Tracking.** The raw measurement: how far the target sits from the image center, first in pixels, then converted to degrees off the lens centerline ("off boresight"). These angles drive the gimbal so it keeps the target centered.
- 3 Dive Correction to FCC.** The product of the whole system. A three-row sum shows the derivation — *where the camera points off the nose, plus where the target sits in the image, equals where the target is relative to the nose* — followed by the exact MAVLink message being streamed and a live status line (streaming / waiting for lock / FCC offline).
- 4 Attitude.** The two attitude sources side by side — FCC and gimbal — and the computed "camera vs nose" difference between them, so any disagreement is visible at a glance.
- 5 System.** Health of the three links (video, gimbal SDK, FCC serial), each with its transport. The header repeats these as green/red dots.

Controls: **left-click** a detection to lock it, **right-click** to release, **Q** to quit. The same correction is also printed to the console once per second for logging.

3 How the error is calculated

The problem: the camera sees the target, but the camera is on a stabilized gimbal that deliberately points away from where the aircraft points. So a target "5° right" *in the image* is not 5° right *of the nose*. The code converts the first into the second in four steps, all in

```
fcc_landing_feedback.py:
```

- 1 **Measure the pixel error.** The tracker gives the target's box; its center minus the image center gives an offset in pixels (`VisionOffset`).
- 2 **Pixels → camera angles — `offset_to_angles()`**. Using the lens field of view ($65^\circ \times 39^\circ$), the pixel offset becomes two angles: how far the target is right of, and below, the camera's own centerline.
- 3 **Find the camera-vs-nose rotation — `mount_relative_euler()`**. The ZR10 reports its pitch and roll against gravity (it self-stabilizes) but its yaw relative to the aircraft's nose. This function combines that report with the FCC's own attitude to strip out the aircraft's tilt, leaving a clean answer to “which way is the camera turned, relative to the airframe?”
- 4 **Rotate into the aircraft's frame — `body_frame_angles()`**. The target's direction is built as a 3-D ray from step 2, rotated by the camera-vs-nose rotation from step 3, and read out as **yaw error** (degrees right of the nose) and **pitch error** (degrees above the nose). This pair is sent to the FCC as `DEBUG_VECT "VIS_ERR"`.

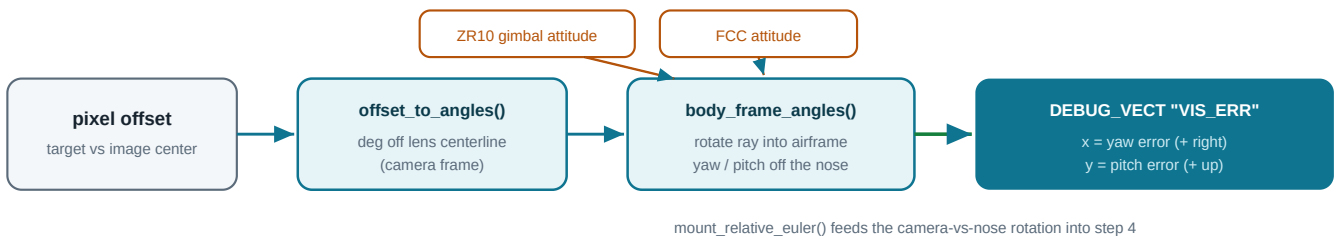


Fig. 2 — The error pipeline. Two attitude sources (amber) are what make the conversion from “in the image” to “off the nose” possible.

Why not just add the angles? The dashboard's sum table carries the note “combined as 3D rotations — not an exact sum”. Yaw measured by a tilted camera is not the same axis as yaw at the nose, so the code multiplies rotation matrices instead of adding degrees. At bench angles the difference is a fraction of a degree; in a steep dive it grows to many degrees — a target 5° right in the image at 60° of camera down-tilt is really about 10° right of the nose. Naive addition would under-steer by half.

4 How the UAV reaches the target

The guidance idea is the same one your eyes use on a football: **keep the nose pointed at the target and keep flying; the path collapses onto it.** The two numbers in `VIS_ERR` are exactly the steering instruction needed to do that at every instant of the dive:

yaw error $+3^\circ$ → steer 3° right · **pitch error -2°** → push 2° down · **both ≈ 0** → nose is on target, hold the dive.

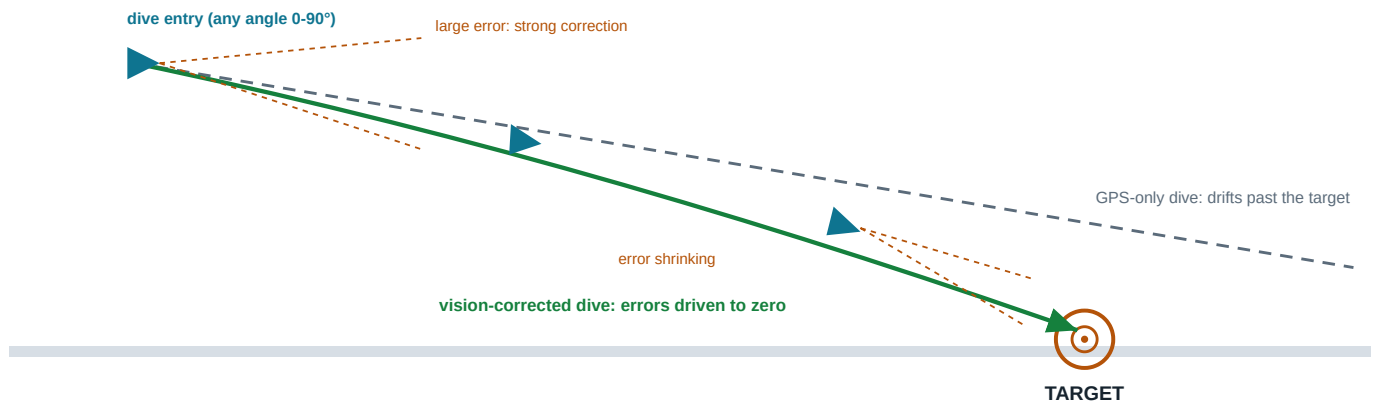


Fig. 3 — Pursuit by error-nulling. Each correction turns the velocity vector a little more onto the target; as the errors shrink to zero the flight path and the line of sight merge.

Because the error is measured **off the nose** — not off the vertical — this works identically for a shallow 20° dive and a near-vertical 85° one. The gimbal keeps the target in view even while the airframe maneuvers, and the mathematics of Section 3 keeps translating what the camera sees into nose-relative corrections regardless of how the two are angled apart.

Demonstration scope. In this build the corrections are *reported, not acted on*: `VIS_ERR` is passive telemetry that any ground station can plot live, which is the point of the demo — proving the camera produces a valid, GPS-independent steering signal. Closing the loop is one step: the FCC (or the existing pursuit module) consumes the same two numbers as rate commands.

5 Code map

File	Role in one line
<code>gui_dashboard.py</code>	Runs everything: five worker threads (video, detection, gimbal, gimbal-steering, FCC/MAVLink) plus the dashboard render loop.
<code>fcc_landing_feedback.py</code>	The error mathematics: <code>offset_to_angles()</code> , <code>mount_relative_euler()</code> , <code>body_frame_angles()</code> .
<code>head_tracker.py</code>	YuNet CNN head detection, click-to-lock matching, and the PID that steers the gimbal.
<code>siyi_zr10.py</code>	Minimal SIYI SDK client: gimbal attitude, centering, rate steering over UDP.

